

PRACTICAL - 3

AIM :Implementation of max-heap sort algorithm

ALGORITHM :

Algorithm :

Input $\leftarrow$ An array A of n elements.
Output $\leftarrow$ Array sorted in ascending order.

Step 1 $\leftarrow$ Start
Step 2 $\leftarrow$ Read n and array A
Step 3 $\leftarrow$ Build a max heap.
Step 4 $\leftarrow$ For $i = n/2 - 1$ down to 0: heapify (arr, n, i);
Step 5 $\leftarrow$ Heapify (arr, n, i);
a. Set largest = i.
b. Set left = 2 * i + 1
c. Set right = 2 * i + 2
d. IF left < n and arr[left] > arr[largest].
Set largest = left.
e. IF right < n and arr[right] > arr[largest].
Set largest = right.
f. IF largest != i,
Swap arr[i] and arr[largest],
Call heapify (arr, n, largest).
Step 6 $\leftarrow$ Perform Heap sort:
For $i = n - 1$ down to 1:
a. Swap arr[0] and arr[i].
b. Call heapify (arr, i, 0).
Step 7 $\leftarrow$ Display the sorted array
Step 8 $\leftarrow$ Stop

PROGRAM :

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
```

```
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap Sort
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;
    int arr[100];

    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted array: ";
```

```
for (int i = 0; i < n; i++)  
    cout << arr[i] << " ";  
  
return 0;  
}
```

Output

```
Enter size of array: 5  
Enter array elements: 23  
45  
12  
2  
3  
Sorted array: 2 3 12 23 45
```

time complexity
best case :

heap sort best case
 for ($i = n/2 - 1$; $i \geq 0$; $i--$) - $O(n)$
 heapify (arr, n, i); - $O(n)$
 for ($i = n - 1$; $i > 0$; $i--$) - n
 { swap (arr[0], arr[i]); - n
 heapify (arr, i, 0); - $O(n \log n)$
 }
 $T(n) = O(n) + O(n) + O(n) + O(n) + O(n \log n)$
 $= O(n \log n)$

average case :

for ($i = n/2 - 1$; $i \geq 0$; $i--$) - $O(n)$
 heapify (arr, n, i); - $O(n)$
 for ($i = n - 1$; $i > 0$; $i--$) - n
 { swap (arr[0], arr[i]); - n
 heapify (arr, i, 0); - $O(n \log n)$
 }
 $T(n) = O(n) + O(n) + O(n) + O(n) + O(n \log n)$
 $= O(n \log n)$

worst case :

```

for (i = n/2 - 1; i >= 0; i--) - O(n)
    heapify(arr, n, i); - O(n)
for (i = n-1; i > 0; i--) - n
{
    swap(arr[i], arr[0]); - n
    heapify(arr, i, 0); - O(n log n)
}
T(n) = O(n) + O(n) + O(n) + O(n) + O(n log n)
      = O(n log n)
    
```